

Data Science Colle... · [Follow publication](#)

You're reading for free via [Shaw Talebi's](#) Friend Link. [Become a member](#) to access the best of Medium.

Member-only story

Machine Learning +

Artificial Intelligence +

Data Science +

Deep Learning +

Neural Networks +

ML Foundations for AI Engineers

A practical guide to teaching computers anything

11 min read · May 9, 2025



Shaw Talebi

[Follow](#)

Listen



Share

AI (as broadly understood today) is built on top of machine learning (ML). Although modern ML models allow developers to create powerful AI apps without understanding their inner workings, this technical debt eventually comes due, causing confusion, bottlenecks, and broken apps. In this guide, I'll cover the essentials of ML that builders need to know to be successful AI engineers.



Image from Canva.

Continue to Medium

[Sign up](#)Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Over the past 6 months, I've taught 100+ (non-ML) engineers how to build custom AI applications with LLMs. Although they can go far (and fast) with their existing technical skills, **most eventually hit a wall**.

The root cause of this is a **lack of knowledge about fundamental ML concepts**. This guide aims to help builders avoid this technical wall by reviewing the core concepts that underlie modern AI development.

Here's an **overview** of what will be covered:

- [Intelligence & Models](#)
- [3 Ways Computers Can Learn](#)
- [Way 1: Machine Learning \(ML\)](#)
- [Way 2: Deep Learning \(DL\)](#)
- [Way 3: Reinforcement Learning \(RL\)](#)
- [Data >](#)

Intelligence & Models

Intelligence boils down to **understanding how the world works**. Whether from a human or a computer, this requires having an internal model of the world.

Put simply, a **model** is **something that lets you make predictions**. The better your model, the more accurate your predictions.

We (i.e. humans) develop our world models by learning from others and our experiences. Computers, it turns out, can learn in much the same way.

3 Ways Computers Can Learn

In traditional software development, computers are given explicit instructions for performing a task. In other words, human

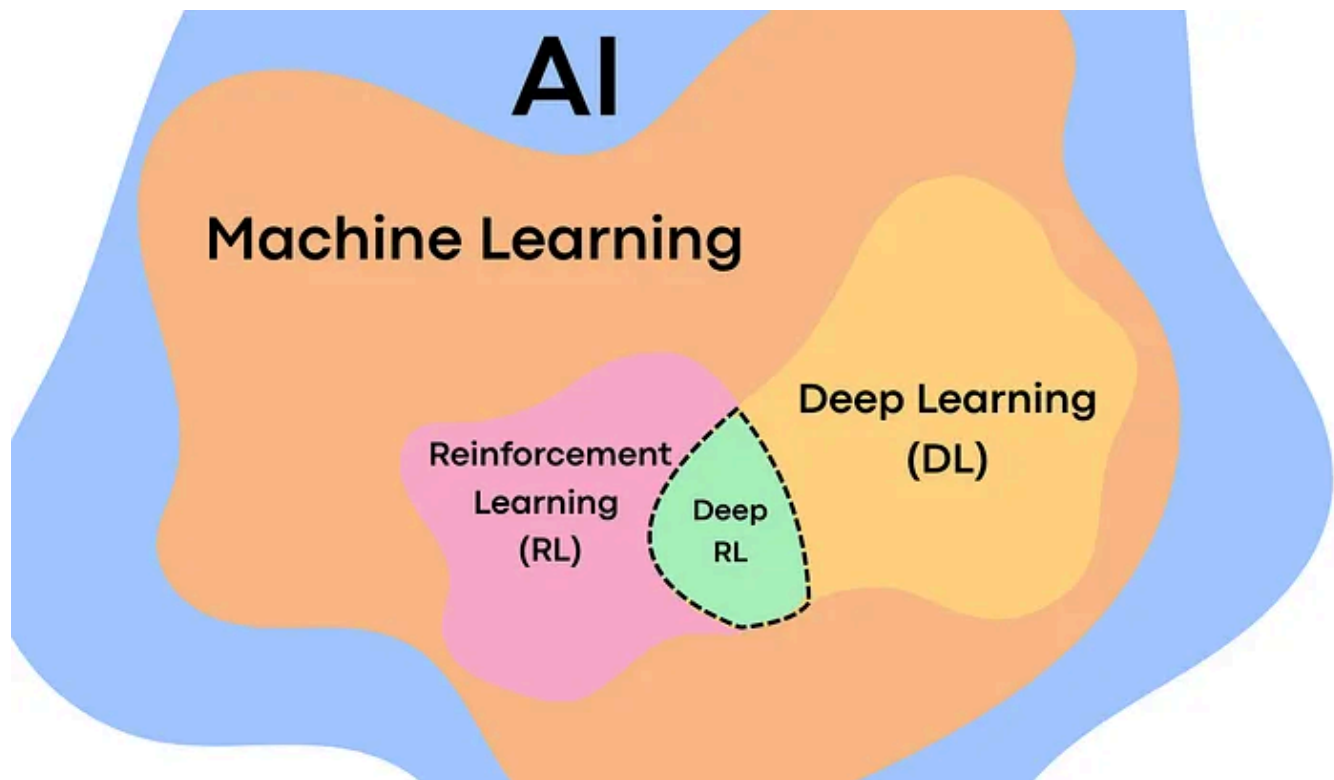
Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

This gives rise to 3 key ways computers can learn about the world. In the following sections, we will define these terms, understand how they're related, and discuss their key concepts.



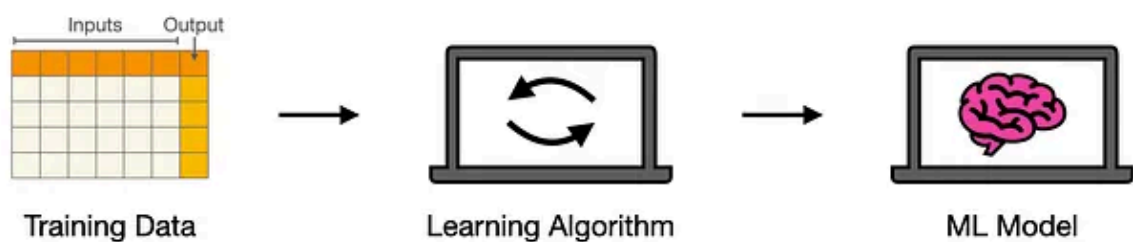
Map of AI. Image by author.

Way 1: Machine Learning

Machine learning (ML) allows computers to learn tasks directly from data [1]. This is the technological foundation for deep learning, reinforcement learning, and all the modern AI innovations we see today.

ML consists of two key phases: training and inference. **Training** is where a model is learned through curated examples [2]. **Inference** involves applying a model to new data to make predictions.

Phase 1: Training



Phase 2: Inference



Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

For example, suppose we want to predict tomorrow's high temperature based on today's. One simple approach is to use a **linear model**, like the one shown below.

$$\text{Prediction } \hat{y} = \text{Input } x \cdot m + b$$

The diagram shows the equation $\hat{y} = mx + b$. Above $\hat{y}$ is the word "Prediction" with a yellow arrow pointing to it. Above x is the word "Input" with a green arrow pointing to it. Below m and b is the word "Parameters" with blue arrows pointing to each. The terms $\hat{y}$, x , m , and b are highlighted in yellow, green, blue, and blue respectively.

$\hat{y}$ = Tomorrow's high temp (prediction)

x = Today's high temp (actual)

m, b = Model parameters

Simple linear model to predict tomorrow's high temperature. Image by author.

Anytime we want to estimate tomorrow's temperature, we simply plug in today's high and compute the result. This mapping of today's temp to tomorrow's is determined by the **parameters** of our model: m and b .

But, *where do these values come from?* Enter training.

Training (Phase 1)

The goal of **training** is to **fit a model's predictions to reality**. At its core, training consists of choosing parameters (e.g. m and b) which minimize the discrepancy between our model's predictions and reality.

We do this by curating a **training dataset**, a collection of input-output pairs that guide model development. This allows us to quantify the discrepancy between model predictions and reality via a so-called **loss function**. Here's one way we can do that.

Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

$$L(m, b) = \left\| \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} - \left(m \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + b \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \right) \right\|^2$$

↑ **Actual Values**
↑ **Predicted Values**

$$\text{Where, } \left\| \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} \right\|^2 = z_1^2 + z_2^2 + \dots + z_n^2$$

Simple loss function for a linear model. Image by author.

Aligning our model to reality, thus comes down to **finding the values of m and b that minimize our loss function**. This, it turns out, is a problem we can solve using math.

Namely, we **compute the gradient of $L(m, b)$** with respect to our parameters, set it equal to zero, and solve for each parameter. Here's what that looks like.

$$\text{Let, } X = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}, \quad \theta = \begin{bmatrix} m \\ b \end{bmatrix}, \quad \text{and} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Inputs
Parameters
Targets

$$\text{Loss Function: } L(\theta) = \|y - X\theta\|^2 = (y - X\theta)^T (y - X\theta) \\ = y^T y - 2\theta^T X^T y + \theta^T X^T X \theta$$

$$\text{Compute gradient of } L(\theta), \quad \nabla_{\theta} L(\theta) = -2X^T y + 2X^T X \theta$$

$$\text{Set gradient equal to 0, } -2X^T y + 2X^T X \theta = 0 \Rightarrow X^T X \theta = X^T y$$

$$\text{Solve for } \theta, \quad \boxed{\theta^* = (X^T X)^{-1} X^T y} \quad \text{Optimal Parameters (for training data)}$$

Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Of course, this linear model is very simple and thus **limited in its ability to explain reality accurately**. The table below summarizes a handful of other popular ML approaches. While their details vary, they all involve this central idea of learning model parameters from real-world data.

Name	Description	Loss Function	Type
Linear Regression	Predicts continuous output by fitting a linear relationship between inputs and output	Mean Squared Error (MSE)	Regression
Logistic Regression	Models the probability of a binary outcome using a logistic (sigmoid) function	Binary Cross-Entropy (Log Loss)	Classification
Decision Tree	Splits data into branches based on feature values to make predictions	Impurity measures (e.g., Gini, Entropy, MSE)	Both
Random Forest	Ensemble of decision trees averaged (regression) or voted (classification)	Same as Decision Tree	Both
XGBoost	Gradient boosting framework that builds trees sequentially to correct prior errors	Customizable; often Log Loss or MSE	Both
SVM (Support Vector Machine)	Finds the optimal hyperplane that separates classes or fits data	Hinge loss (classification), Epsilon-insensitive loss (regression)	Both

Table of popular supervised learning techniques. Image by author.

Way 2: Deep Learning

Although the **traditional ML techniques** described above can perform a wide range of tasks, they **require domain expertise (and experimentation) to pick the right model inputs**. For example, when I worked in credit risk, we'd pay bureaus for credit scores, i.e., metrics built on decades of domain expertise and carefully crafted data, because they consistently predicted default risk.

This process of **defining a helpful set of input variables** for an ML model is called **feature engineering**, and it can make or break an ML model's performance. However, the importance of feature engineering began to wane in 2012 with the release of AlexNet and the rise of deep learning [3].

AlexNet demonstrated that a **deep neural network (DNN)** could outperform other ML approaches without employing any explicit feature engineering [3]. Images with labels were passed to the DNN, which **learned helpful features on its own**. This is the essence (and power) of deep learning.



Deep Neural Nets

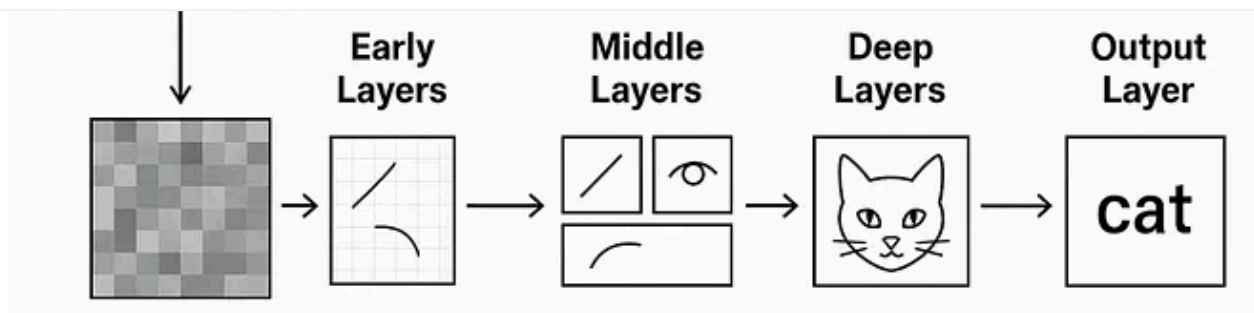
Open in app ↗

Sign up

Sign in

Medium

Search



Continue to Medium

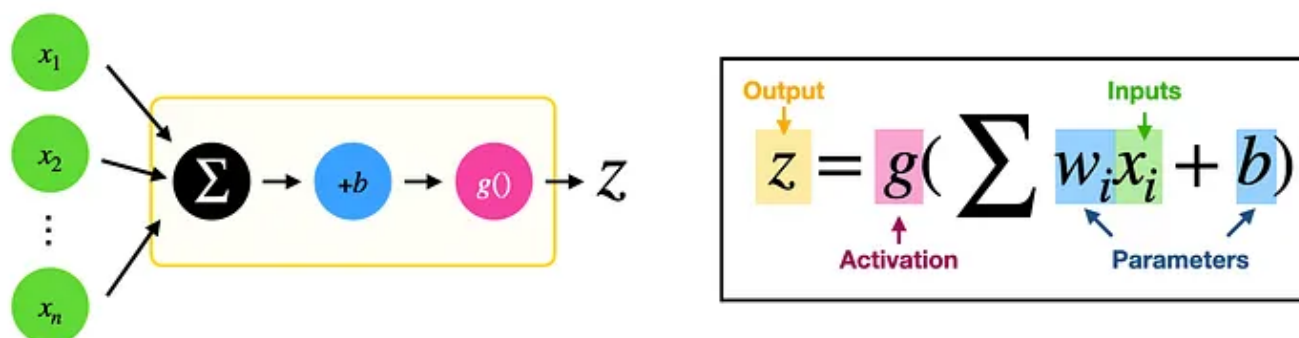
Sign up

Already have an account? Sign in

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Deep learning involves training neural networks that learn *optimal* features for an arbitrary task. The fundamental building block of a neural network (NN) is a neuron, which is described in the figure below.

(Artificial) Neuron

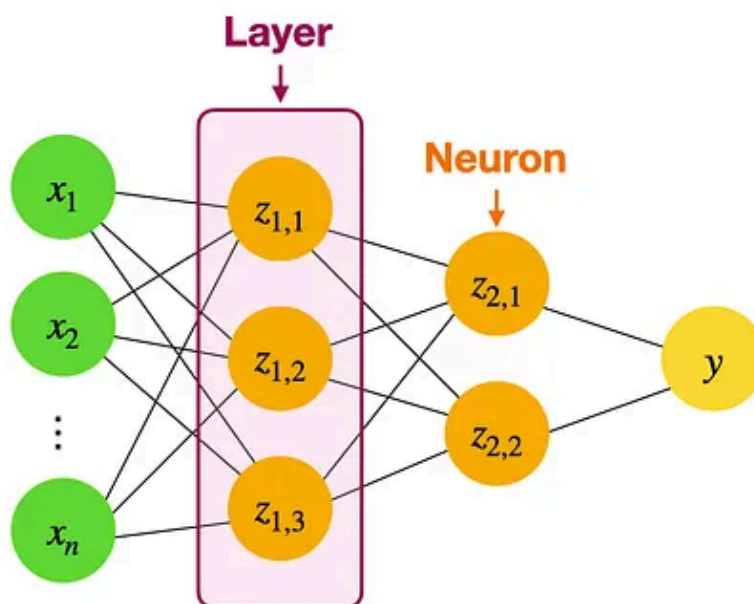


An (artificial) neuron. (Left) cartoon of neuron's information flow. (Right) the math of a neuron.

Notice that the neuron is a lot like the linear model we used to predict tomorrow's temperature. The key difference, however, is that we pass it through a **nonlinear activation function**. This is a critical step because it **allows neural networks to approximate any continuous function** (given enough neurons and layers). In other words, you can practically train an NN to do anything.

We can **stack multiple neurons** together to form a **layer** and then **combine multiple layers** in a sequence to form a **network**. The key intuition of deep learning is that layers in an NN build upon previous ones to derive increasingly complex features for performing the given task, as illustrated in the cat image above.

Deep Neural Network



A simple (feed-forward) neural network with fully connected layers. Image by author.

Continue to Medium

Sign up

Already have an account? Sign in

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Neurons

Name

Description

Vanilla

Basic unit computing weighted sum + activation; used in FFNNs and CNNs

LSTM

Advanced neuron with memory and gates for long-term dependencies in sequences

Layers

Activations

Name

Description

ReLU

$\max(0, x)$; fast, widely used in deep networks

Sigmoid

S-shaped, output in $(0, 1)$; used in binary classification

Tanh

Like sigmoid but centered at 0; output in $(-1, 1)$

Softmax

Outputs a probability distribution; used in final layer of multi-class classification

Name	Description
Fully Connected	Standard layer where each neuron connects to all inputs
Recurrent	Maintains memory across timesteps; used in RNNs, LSTMs
Convolutional	Extracts spatial features using filters; used in image data
Attention	Computes weighted importance of different inputs; key in Transformers
Pooling	Downsamples spatial data; used in CNNs to reduce size and noise
Normalization	Stabilizes training by normalizing activations; includes BatchNorm, LayerNorm
Dropout	Randomly deactivates neurons during training to prevent overfitting

Networks

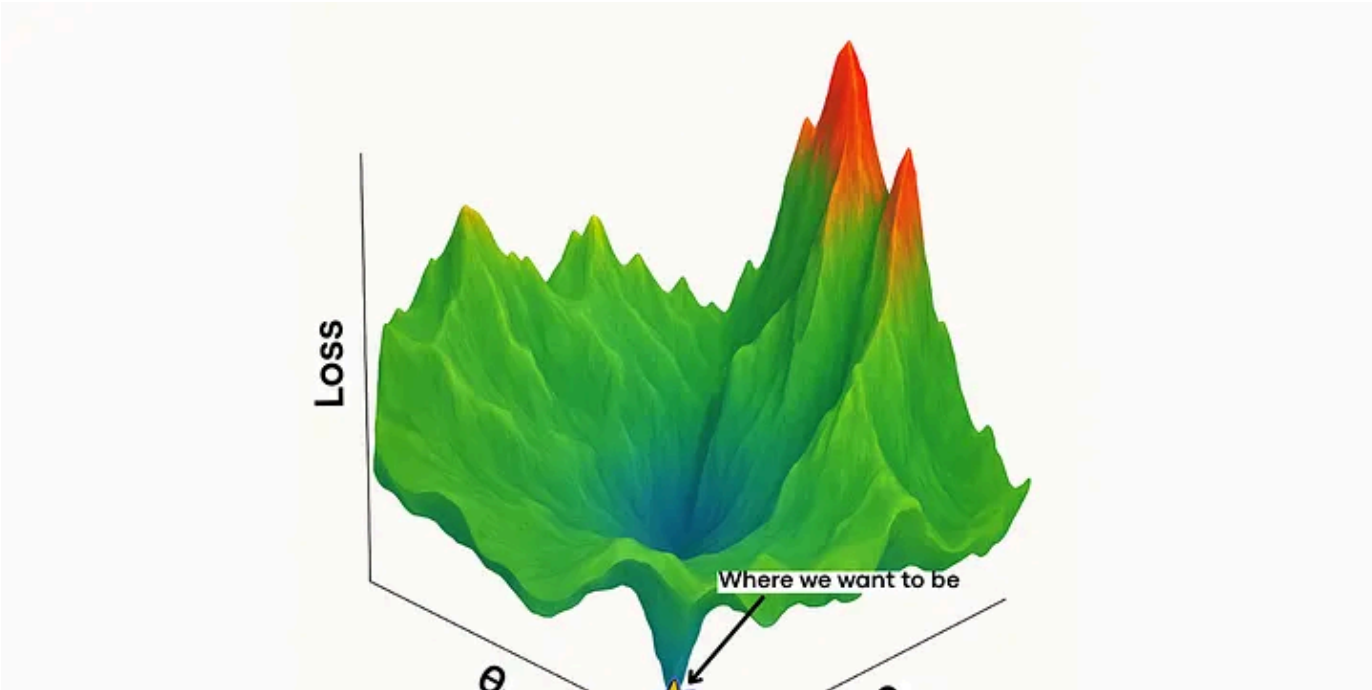
Name	Description
Feedforward (FFNN)	Basic architecture with no loops; used for static input-output tasks
RNN	Handles sequential data using recurrence; remembers previous inputs
CNN	Uses convolutions to process grid-like data such as images
Transformer	Uses self-attention to model sequences without recurrence; state-of-the-art in NLP & beyond

Zoo of neural network components and architectures. Image by author in collaboration with ChatGPT.

Training Neural Networks

Although the non-linearities and depth of a DNN allow it to approximate practically any function, it also complicates the training process. Unlike the linear model we saw earlier, the loss function for a DNN is generally non-convex, meaning we can't solve for the best model parameters using algebra (like we did earlier).

Instead, we need to use an algorithm (i.e. an optimizer) to explore parameter choices until we find those that give us the smallest loss value. A helpful mental picture of this is shown below, where we have a loss function (L) that depends on parameters (e.g. θ_1 and θ_2).



Continue to Medium

Sign up

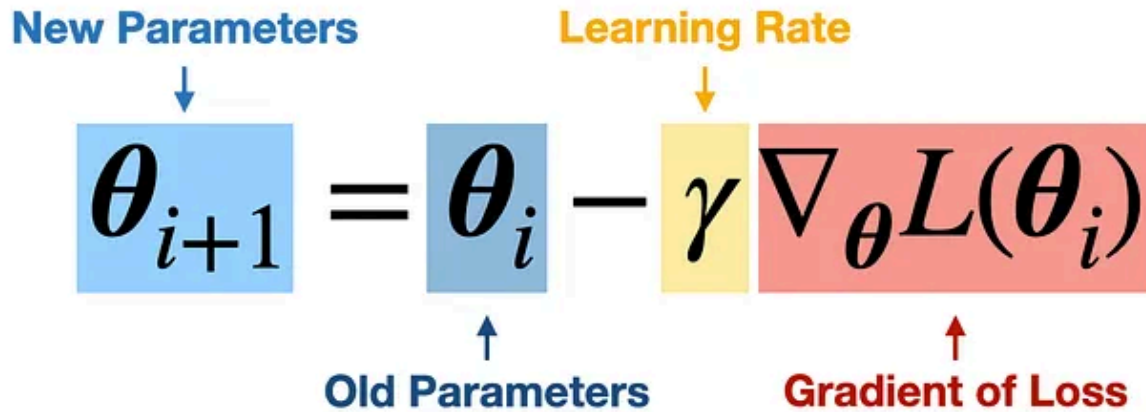
Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Therefore, we can use them as guides to traverse toward the bottom of this perilous terrain.

In mathematical terms, this translates to an update rule for our model parameters at each step in the training process. The equation is given below.

Gradient Descent


$$\theta_{i+1} = \theta_i - \gamma \nabla_{\theta} L(\theta_i)$$

Standard form of gradient descent. Image by author.

In true gradient descent, parameter updates are based on the loss averaged over the entire training dataset. However, this tends to create convergence issues, so several variations have been developed (see table below).

Common Optimizers

Name	Description	Data Sampled	Key Hyperparameters
Gradient Descent	Updates model parameters using the gradient computed on the entire dataset.	Full dataset (batch)	Learning rate (γ)
Stochastic Gradient Descent (SGD)	Updates parameters using the gradient from a single random example.	One data point at a time	Learning rate (γ)
Mini-Batch Gradient Descent	Combines efficiency and noise reduction by using small random batches.	Batch of data (e.g., 8, 16)	Learning rate (γ), batch size
Adam (Adaptive Moment Estimation)	Combines momentum and RMSProp by maintaining adaptive learning rates.	Mini-batches (usually)	Learning rate (γ), β_1 (momentum), β_2 (RMS), ϵ (stability constant)

Table of common optimizers. Image by author in collaboration with ChatGPT.

Hyperparameters

Hyperparameters are values that guide the training process. Unlike parameters, they are not learned during training—they are set manually by the developer or tuned using a so-called validation dataset.

Here are the most common ones you'll see when training neural nets.

Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Common Hyperparameters

Name	Description
Epoch	Number of times the entire dataset is passed through the model during training
Learning Rate (γ)	Controls how much to adjust weights with respect to the gradient
Batch Size	Number of samples used to compute each update to the model's weights
Dropout	Fraction of neurons randomly set to 0 during training to prevent overfitting

Table of common hyperparameters. Image by author in collaboration with ChatGPT.

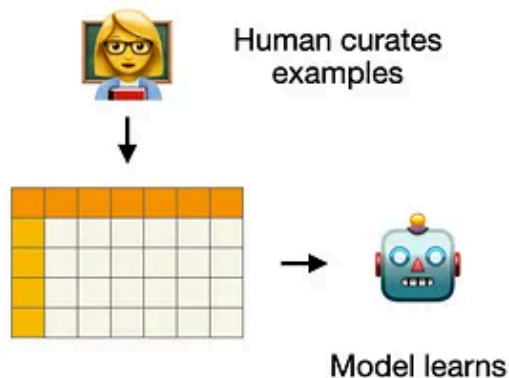
Way 3: Reinforcement Learning

So far, we've focused on **machine learning approaches that learn from explicit examples**, e.g., predicting weather using past data or classifying images given labelled examples. This is analogous to how we might learn from a teacher or mentor who shows us how to perform a specific task.

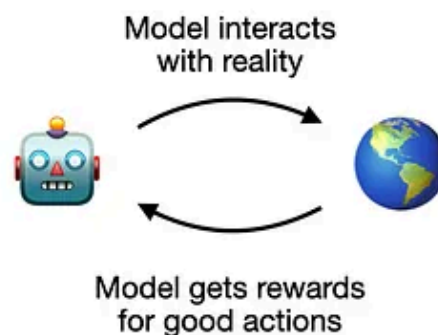
However, **this is not the only way to learn**. In fact, much of the time, we don't learn from others but from real-world experience. For example, I didn't learn how to ride a bike from watching others do it. Instead, I continuously tried, failed, and tried again until I eventually got it.

This is the key idea behind **reinforcement learning (RL)**, where models learn through trial and error.

Supervised Learning



Reinforcement Learning



[Continue to Medium](#)

[Sign up](#)

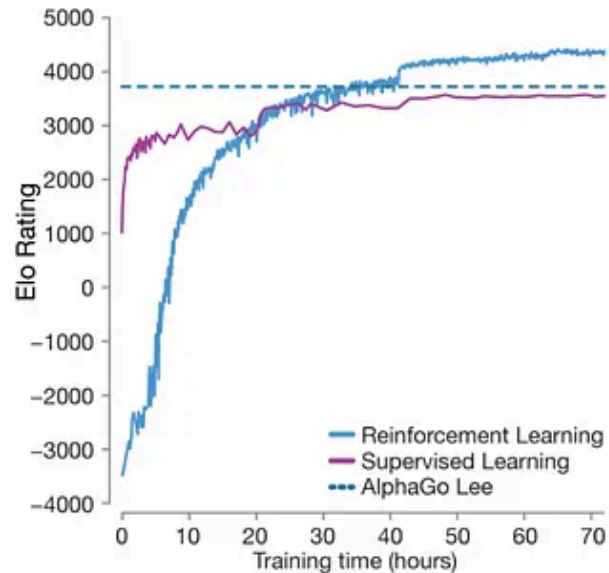
Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

The promise of RL is that **models are not bound by human labelling or expertise**. A notable example is AlphaGo, a deep neural network trained via RL to play the game of Go.

Rather than learning from expert players, AlphaGo learned from playing itself and using rewards like winning the game. Since the model wasn't trying to mimic the experts, it **discovered its own novel approaches to the game and eventually surpassed human grandmasters** [4].

RL surpasses grandmaster performance



Silver, D., Huang, A., Maddison, C. et al. Mastering the game of Go with deep neural networks and tree search. *Nature* 529, 484–489 (2016). <https://doi.org/10.1038/nature16961>

ELO ratings of DNNs trained via reinforcement learning (RL) and supervised learning (SL) compared with a Go grandmaster. Plot from [4].

How does it work?

While reinforcement learning (RL) extends beyond deep learning, its most **impactful applications often rely on DNNs**. These include aligning ChatGPT to human preferences through RLHF (reinforcement learning from human feedback) and more recent innovations like OpenAI's o1 model and deep research mode.

However, a key difference between DNNs trained via RL and those trained with supervised learning (i.e. what we saw in the previous section) lies in the loss function. In supervised learning, we train against explicit task examples—there's a clear ground truth to compare predictions against.

In RL, that ground truth doesn't exist. Instead, **we define a reward signal that indirectly guides learning**. The model updates its parameters to favor behaviors that lead to positive feedback and avoid those that don't. While there are several ways to implement this, most modern approaches are based on the REINFORCE algorithm [5].

Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

REINFORCE Objective

Average over episode Reward at t

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) \cdot R_t \right]$$

Objective Model Output

where, $\pi_{\theta}(a_t | s_t) =$ Probability of taking action a_t given state s_t ,
based on model parameters

$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$ Full trajectory of
states and actions

$T =$ Number of time steps in τ

Fundamental objective function used in RL. Image by author.

The **blue term** is our RL model's output (i.e. the probability of various actions in a given state). The **green term** represents the reward (i.e. how good a particular action was). The goal is to make their product as big as possible because that means our model has a high probability of performing actions that generate a high reward.

To do this, we want to find the model parameters that **maximize the objective function** (J). Again, we can use our handy gradient here. However, since we want to find the parameters yielding the biggest value of J , our parameter update rule has a plus sign instead of a minus (aka Gradient Ascent).

Gradient Ascent

New Parameters

Learning Rate

$$\theta_{k+1} = \theta_k + \gamma \nabla_{\theta} J(\theta_k)$$

Continue to Medium

Sign up

Already have an account? Sign in

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

Of course, RL has evolved a lot since the release of REINFORCE in 1992 [5]. Modern techniques improve upon this core idea to improve training stability and efficiency.

Here is a table summarizing a few popular approaches.

Name	Key Idea	Objective Function
REINFORCE	Monte Carlo policy gradient using full returns.	$J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t s_t) \cdot R_t]$
TRPO	Constrain policy updates with a KL-divergence trust region to ensure stable improvements.	$J(\theta) = \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t s_t)}{\pi_{\theta_{old}}(a_t s_t)} \cdot A_t \right]$ s.t. $\mathbb{E}_t \left[D_{KL}(\pi_{\theta_{old}}(\cdot s_t) \parallel \pi_{\theta}(\cdot s_t)) \right] \leq \delta$
PPO	Approximates TRPO using a clipped surrogate objective for practical stability and performance.	$J(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t) \right]$, where $r_t(\theta) = \frac{\pi_{\theta}(a_t s_t)}{\pi_{\theta_{old}}(a_t s_t)}$
GRPO	Encourages exploration by optimizing relative advantages within action groups, not globally.	$J(\theta) = \mathbb{E}_{q \sim \pi_{\theta}} \left[\frac{1}{G} \sum_{g=1}^G \frac{1}{ a_g } \sum_{a \in a_g} \left\{ \min \left[\frac{\pi_{\theta}(a_{i,j} q, o_{i,j})}{\pi_{\theta_{old}}(a_{i,j} q, o_{i,j})} \hat{A}_{i,j}, \text{clip} \left(\frac{\pi_{\theta}(a_{i,j} q, o_{i,j})}{\pi_{\theta_{old}}(a_{i,j} q, o_{i,j})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,j} \right] - \beta D_{KL}[\pi_{\theta} \parallel \pi_{ref}] \right\} \right]$

Summary table of various popular RL algorithms. Image by author.

Data >

Since we've talked a lot about ML techniques and algorithms, this might give you the impression that this is what ML is all about. While these are important, **what's more critical are the data you use to train ML models.**

No amount of algorithmic sophistication or hyperparameter tuning can solve for bad data. If you're struggling to improve your ML model's performance, odds are bad data is the culprit.

What makes data good?

There are **two properties of good training data: quantity and quality.**

All things equal, more data is better than less data. Insufficient data volume is the leading cause of **overfitting**, i.e., when your **model memorizes your training dataset** and thus isn't helpful for making new predictions.

There is, however, one case where more data is not better — when you have poor data quality. Summarized by the saying, *"garbage in, garbage out"* (no matter how much of it you've got).

Data Quality

Data quality comes down to **two things: accuracy and diversity.**

Accuracy describes **how correct your data is**. For example, if a customer's age is recorded as 35 when it's actually 53, that's inaccurate data. Another is if someone's income is actually \$12k/mo but is recorded as \$12k/yr!

Diversity describes **how well your data represents the scenarios in which you want your model to work**. For example, if you are trying to predict customer churn for a SaaS product, a model trained only on yearly premium users may not generalize well to monthly pro users. Or, if trying to classify the sentiment of customer reviews, training only on X posts may not have good results.

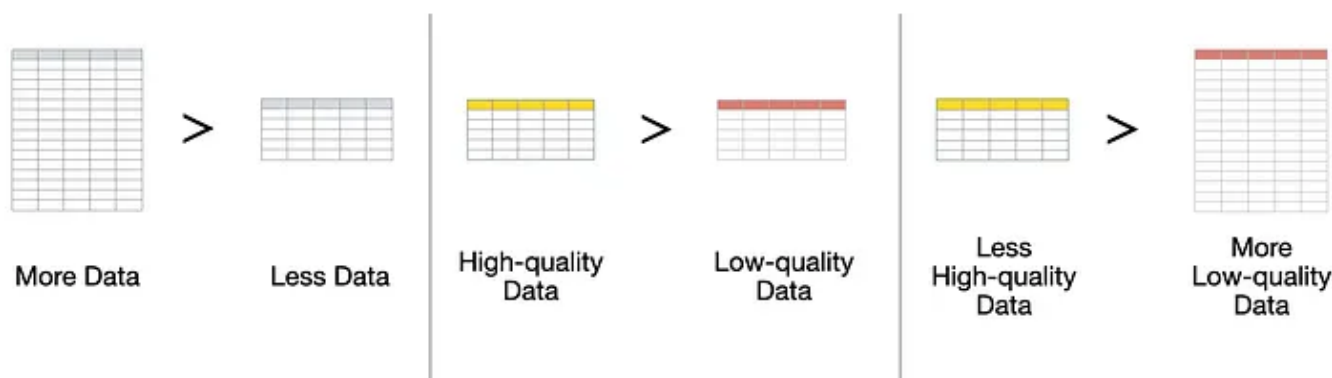
Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).

What makes data “good”?



What makes good data quantity and quality? Image by author.

Conclusions

Solving problems requires having an accurate model of the world. Machine learning (ML) gives computers a way to align models to reality using data and math. This differs from traditional software development because programmers do not need to write explicit instructions for a task; instead, they curate high-quality examples from which the computer can learn.

While there is much that I couldn't cover in this short guide, I hope it clarified how ML works and gave you the confidence to continue learning on your own.

Toward that end, if you have any questions or suggestions for future topics, please let me know in the comments :)

🔗 Join 1,000+ AI builders and get instant access to 30 AI project ideas: <https://the-data-entrepreneurs.kit.com/shaw>

References

- [1] [MIT Introduction to Deep Learning](#).| 6.S191
- [2] [The Royal Society](#). *Machine Learning: The Power and Promise of Computers That Learn by Example*. The Royal Society, 2017. <https://royalsociety.org/~media/policy/projects/machine-learning/publications/machine-learning-report.pdf>
- [3] Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 1097–1105.
- [4] Silver, D., Huang, A., Maddison, C. *et al*. Mastering the game of Go with deep neural networks and tree search. *Nature* 529, 484–489 (2016). <https://doi.org/10.1038/nature16961>
- [5] Williams, R.J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach Learn* 8, 229–256 (1992). <https://doi.org/10.1007/BF00992696>

Machine Learning +

Artificial Intelligence +

Data Science +

Deep Learning +

Neural Networks +

DSC

Continue to Medium

Sign up

Already have an account? [Sign in](#)

By clicking "Sign up", you accept Medium's [Terms of Service](#) and [Privacy Policy](#).